

LEGALEASE AI

AI-Assisted Legal Document Generation System

PHASE 7 – PROJECT DOCUMENTATION

Technical Report • Implementation • Testing • Evidence

Submitted By

Team Members

- 1. SATHVIKA V S**
- 2. SUMATHI**
- 3. RITHIKA P**
- 4. SINDHUJA**

Department of Computer Science

Shrimati Indira Gandhi College

Affiliated with Bharathidasan University

Academic Year: 2026

Document Guide

This document is structured as a complete technical report for LegalEaseAI. It combines system explanation with implementation evidence from the supplied Visual Studio Code screenshots. Screenshots of the running frontend could not be supplied in this session, so dedicated placeholders are retained for those evidence points.

Section	Topic
1	Introduction
2	Project Overview
3	Problem Statement
4	Objectives
5	Proposed Solution
6	System Features
7	Technology Stack
8	System Architecture
9	System Workflow
10	Module Description
11	Project Structure
12	Implementation Details
13	Testing Summary
14	Screenshots / Evidence
15	Installation and Execution
16	Project Outcomes
17	Future Enhancements
18	Conclusion

1. Introduction

LegalEaseAI is an AI-assisted legal document generation application designed to combine a web-based user interface, a FastAPI backend, request validation, an AI generation layer, and document-formatting utilities. The supplied project implementation shows a modular Python application in which the frontend communicates with backend routes and the backend delegates AI-oriented document generation to a dedicated Gemini service.

The project follows a separation-of-concerns approach. Startup logic is kept in `run.py`, backend initialization is defined in `LegalEaseAPI/main.py`, request handling is organized in `LegalEaseAPI/routes.py`, AI generation is isolated in `ai_core/gemini_generator.py`, document formatting is handled in `utils/formatter.py`, and the user-facing application is implemented in `frontend/app.py`. A dedicated stylesheet in `frontend/static/style.css` provides the visual theme.

The documentation therefore presents LegalEaseAI not simply as a collection of source files, but as an integrated software system. Each major implementation decision is connected to its role in the overall workflow and, where available, supported by a project screenshot.

Evidence principle

The screenshots included in this report are treated as implementation evidence. Claims are limited to what is visible in the supplied project structure, source-code views, and successful startup terminal output.

2. Project Overview

LegalEaseAI is organized as a local web application with two principal runtime-facing components: a FastAPI backend and a Streamlit frontend. The startup script launches the backend through Uvicorn and starts the frontend service. The terminal evidence supplied for the project shows the backend at port 8000 and the frontend at port 8501.

The AI layer is represented by `GeminiDocumentGenerator`. The supplied source view shows that this class loads environment variables, obtains the `GEMINI_API_KEY` configuration value, and configures a Gemini model. The backend routes create an instance of this generator and expose request-oriented functionality through an API router.

The formatting layer provides text sanitization and output-cleaning functions and imports both `python-docx` and `ReportLab` components, indicating that the application contains utilities for structured document output. The exact final UI output is reserved

for the frontend evidence section because those browser screenshots were not available in the current session.

2.1 High-Level Project Components

Startup layer — run.py coordinates application startup and defines the backend/frontend ports.

Backend layer — FastAPI provides the application object, middleware configuration, and route registration.

API layer — routes.py defines API routing, request models, validation, and Gemini service initialization.

AI layer — gemini_generator.py encapsulates Gemini configuration and AI-assisted document generation logic.

Formatting layer — formatter.py sanitizes and prepares generated text for document-oriented output.

Frontend layer — frontend/app.py initializes the Streamlit application and its page configuration.

Styling layer — style.css defines the Luna Blue + Green visual system used by the frontend.

3. Problem Statement

Preparing legal-style documents manually can require repeated drafting, formatting, editing, and organization. A user may need to provide document-specific information, transform that information into a structured draft, and then prepare the resulting content for a usable document format. Repeating these steps manually can make the process time-consuming and inconsistent.

The project addresses this software-process problem by creating a centralized application flow in which user-facing input can be sent to a backend API, validated using structured request models, processed through an AI generation component, and passed through output-formatting utilities.

From a software-engineering perspective, the problem is also one of maintainability: mixing UI code, API logic, AI integration, and formatting code in a single file makes a project harder to test and extend. LegalEaseAI separates these responsibilities into modules so that each major concern has a defined location.

4. Objectives

The main objective is to develop a modular AI-assisted application for generating structured legal document drafts through a web interface. The implementation evidence supports the following project objectives:

Provide a user-facing Streamlit interface for interacting with the application.

Provide a FastAPI backend for structured API-based communication.

Validate incoming document-generation requests using Pydantic models and field constraints.

Integrate a Gemini-based generation component through a dedicated service class.

Load sensitive configuration such as the Gemini API key from environment variables rather than hard-coding it in source code.

Clean and sanitize generated text before document-oriented formatting.

Separate application startup, backend routing, AI generation, formatting, frontend presentation, and styling into independent modules.

Provide a repeatable local execution process through `run.py`.

Maintain a consistent visual identity using the project's Luna Blue + Green stylesheet.

5. Proposed Solution

The proposed solution is a layered web application. At the presentation layer, Streamlit provides the user interface. The backend layer is implemented using FastAPI and exposes API routes. The request layer uses Pydantic models to define and validate incoming data. The AI layer uses GeminiDocumentGenerator to communicate with the configured Gemini model. Finally, the formatting layer prepares generated text for document output.

5.1 Solution Characteristics

Modular — each major responsibility is implemented in a dedicated package or module.

Configurable — environment variables are loaded through `python-dotenv`.

API-driven — the frontend can communicate with the backend through HTTP requests.

Validation-aware — request fields include explicit constraints in the visible `routes.py` implementation.

AI-enabled — the project includes a dedicated Gemini document-generation class.

Output-oriented — formatting utilities prepare generated content for document use.

Maintainable — the project structure makes individual layers easier to locate and modify.

5.2 Scope of the Current Evidence

The supplied screenshots establish the architecture, startup behavior, and major implementation modules. They do not include the final browser interface or a generated-document result. Those screenshots should be added later in Section 14 and the corresponding feature descriptions can then be expanded with exact UI evidence.

6. System Features

6.1 Web-Based User Interface

The frontend is implemented with Streamlit. The visible `app.py` source configures a centered page layout and loads the project environment. The browser-based interface is intended to provide the interaction point for document-generation requests.

6.2 API Backend

FastAPI is used as the backend framework. `main.py` creates the FastAPI application, configures CORS middleware, and includes the API router.

6.3 Request Validation

`routes.py` imports Pydantic `BaseModel`, `Field`, and `field_validator`. The visible `DocumentRequest` model applies a minimum and maximum length constraint to `document_type`, providing a structured validation layer.

6.4 Gemini AI Integration

`gemini_generator.py` defines `GeminiDocumentGenerator` and loads the Gemini API key from the environment. The source describes the component as the LegalEase AI Core and identifies the Gemini model as the generation engine.

6.5 Error Handling

The visible `routes.py` initialization wraps Gemini generator creation in a `try/except` block and records `GEMINI_READY` and `GEMINI_ERROR` state. This provides an explicit way to represent AI-service initialization success or failure.

6.6 Text Sanitization and Formatting

`formatter.py` provides `sanitize_text` and `format_document_output` functions. The visible code strips surrounding whitespace and removes unwanted Markdown code-fence patterns before continuing with document formatting.

6.7 Document-Oriented Output Support

`formatter.py` imports `python-docx`'s `Document` and `ReportLab A4/canvas` components, providing the implementation foundation for structured document output.

6.8 Consistent Visual Theme

`style.css` defines a Luna Blue + Green palette with navy, blue, teal, and green variables and success-state styling.

7. Technology Stack

Technology / Component	Role	Evidence in Project
Python	Primary programming language	All visible implementation files use Python.
FastAPI	Backend web/API framework	<code>main.py</code> imports FastAPI and creates <code>app = FastAPI(...)</code> .
Uvicorn	ASGI server for backend	<code>run.py</code> builds a Uvicorn command for <code>LegalEaseAPI.main:app</code> .
Streamlit	Frontend web application framework	<code>frontend/app.py</code> imports streamlit as <code>st</code> and calls <code>st.set_page_config</code> .
Pydantic	Request data modeling and validation	<code>routes.py</code> imports <code>BaseModel</code> , <code>Field</code> , and <code>field_validator</code> .
Google Gemini / google-genai	AI generation service	<code>gemini_generator.py</code> imports <code>google.genai</code> and defines <code>GeminiDocumentGenerator</code> .
python-dotenv	Environment configuration	<code>gemini_generator.py</code> and <code>frontend/app.py</code> load <code>.env</code> values.
python-docx	DOCX-oriented document handling	<code>formatter.py</code> imports <code>Document</code> .

ReportLab	PDF-oriented output support	<i>formatter.py imports A4 and canvas.</i>
CSS	Frontend styling	<i>frontend/static/style.css defines the Luna Blue + Green theme.</i>

8. System Architecture

The visible implementation supports a layered architecture in which startup orchestration, presentation, API routing, AI generation, and formatting are separated. The architecture reduces coupling by allowing the frontend, API, AI service, and formatting utility to be developed as distinct concerns.

8.1 Logical Architecture

Presentation Layer

Streamlit frontend in *frontend/app.py*. Responsible for page configuration and user-facing interaction.

Application/API Layer

FastAPI application in *LegalEaseAPI/main.py*. Responsible for creating the backend application, middleware, and router registration.

Request Layer

LegalEaseAPI/routes.py. Defines API routing and Pydantic request structures.

AI Service Layer

ai_core/gemini_generator.py. Loads configuration and initializes the Gemini document-generation service.

Formatting Layer

utils/formatter.py. Sanitizes generated text and prepares it for document-oriented output.

Execution Layer

run.py. Starts the backend and frontend services and defines the local ports.

Architecture flow

User Interface → API Request → Validation → Gemini Document Generator → Text Formatting → Document-Oriented Output → User Access

10. Module Description

10.1 run.py

Acts as the application launcher. It imports subprocess, sys, and time; defines BACKEND_HOST, BACKEND_PORT, and FRONTEND_PORT; constructs the Uvicorn backend command; and coordinates the service startup.

10.2 LegalEaseAPI/main.py

Creates the FastAPI application, assigns the title 'LegalEase API', provides a backend description and version, configures CORS middleware, and includes the API router.

10.3 LegalEaseAPI/routes.py

Creates an APIRouter, imports the Gemini document generator, initializes the Gemini service with explicit readiness/error state, and defines Pydantic request structures.

10.4 ai_core/gemini_generator.py

Provides the GeminiDocumentGenerator class. It loads environment variables, retrieves GEMINI_API_KEY, validates that the key exists, and configures the Gemini model.

10.5 utils/formatter.py

Provides text sanitization and output-formatting functions. It removes surrounding whitespace and cleans accidental Markdown code fences. It imports DOCX and ReportLab components for document output.

10.6 frontend/app.py

Initializes the Streamlit application, resolves the project root, loads the .env file, and configures the Streamlit page title, icon, layout, and sidebar state.

10.7 frontend/static/style.css

Defines the project's Luna Blue + Green design system through CSS variables and styles the Streamlit page background and other interface elements.

11. Project Structure

The project is organized into dedicated directories for AI logic, frontend presentation, backend API logic, utilities, and the Python virtual environment. The structure visible in Visual Studio Code provides a clear map of the major source components.

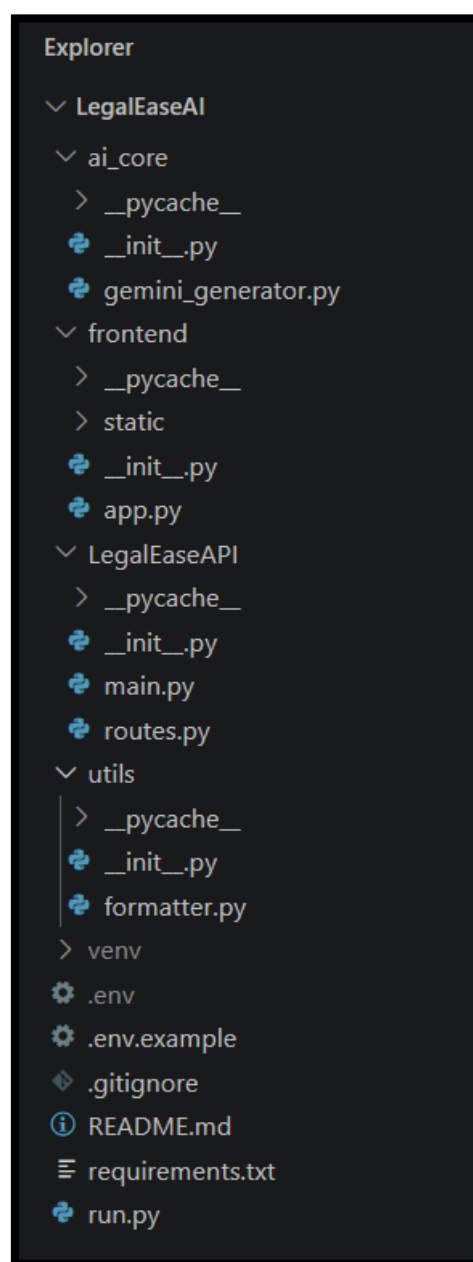


Fig. 11.1 – Complete LegalEaseAI project structure in Visual Studio Code

The `ai_core` directory contains the Gemini generation component. The frontend directory contains `app.py` and static resources. `LegalEaseAPI` contains the backend application and routes. The `utils` directory contains `formatter.py`. The project root contains environment/configuration files, `README.md`, `requirements.txt`, and `run.py`. The `venv` directory represents the project's local Python virtual environment.

11.1 Structure Summary

- I. ai_core/ — AI and Gemini-related functionality.*
- II. frontend/ — Streamlit frontend and static styling resources.*
- III. LegalEaseAPI/ — FastAPI application and route definitions.*
- IV. utils/ — reusable document/text formatting utilities.*
- V. venv/ — local Python virtual environment.*
- VI. .env / .env.example — environment configuration files.*
- VII. requirements.txt — project dependency declaration.*
- VIII. README.md — project documentation entry point.*
- IX. run.py — application startup script.*

12. Implementation Details

This section documents the implementation evidence file by file. Each screenshot is positioned immediately after the explanation of the responsibility it demonstrates, making the report easier to audit and understand.

12.1 Application Startup and Service Initialization

`run.py` establishes the local runtime configuration. It defines the backend host as `127.0.0.1`, the backend port as `8000`, and the frontend port as `8501`. The backend command invokes the current Python executable with the Uvicorn module and points Uvicorn to `LegalEaseAPI.main:app`. The script therefore provides a single entry point for starting the application services.

```
main.py X
LegalEaseAPI > main.py > ...
1  from fastapi import FastAPI
2  from fastapi.middleware.cors import CORSMiddleware
3
4  from LegalEaseAPI.routes import router
5
6
7  app = FastAPI(
8      title="LegalEase API",
9      description=(
10         "Backend API powering AI-assisted "
11         "legal document generation."
12     ),
13     version="1.0.0"
14 )
15
16
17 # =====
18 # CORS
19 # =====
20
21 app.add_middleware(
22     CORSMiddleware,
23     allow_origins=["*"],
24     allow_credentials=True,
25     allow_methods=["*"],
26     allow_headers=["*"],
27 )
28
29
30 # =====
31 # ROUTES
32 # =====
33
```

Fig. 12.2 – FastAPI backend initialization, CORS configuration and route registration

12.3 API Routing and Request Validation

routes.py is responsible for the API-facing request layer. The visible source imports APIRouter, HTTPException, status, and Pydantic validation tools. It initializes GeminiDocumentGenerator and stores explicit readiness and error states. The DocumentRequest model uses a document_type field with visible minimum and maximum length constraints, providing structured input validation before deeper processing.

```
main.py routes.py X
LegalEaseAPI > routes.py > generate_legal_document
1  from fastapi import APIRouter, HTTPException, status
2  from pydantic import BaseModel, Field, field_validator
3
4  from ai_core.gemini_generator import GeminiDocumentGenerator
5
6
7  router = APIRouter()
8
9
10 # =====
11 # GEMINI SERVICE
12 # =====
13
14 try:
15     gemini_generator = GeminiDocumentGenerator()
16     GEMINI_READY = True
17     GEMINI_ERROR = None
18
19 except Exception as e:
20     gemini_generator = None
21     GEMINI_READY = False
22     GEMINI_ERROR = str(e)
23
24
25 # =====
26 # REQUEST MODEL
27 # =====
28
29 class DocumentRequest(BaseModel):
30
31     document_type: str = Field(
32         ...,
33         min_length=2
```

Fig. 12.3 – API routing, Gemini service initialization and request validation

12.4 AI-Based Legal Document Generation

gemini_generator.py contains the core AI service class, *GeminiDocumentGenerator*. It loads environment variables using *load_dotenv*, retrieves *GEMINI_API_KEY* from the environment, and raises a clear *ValueError* if the required key is missing. The source also configures the Gemini model through an environment-based model name, keeping service configuration outside the main application logic.

```
gemini_generator.py X
ai_core > gemini_generator.py > GeminiDocumentGenerator > __init__
1  import os
2  import time
3
4  from dotenv import load_dotenv
5  from google import genai
6  from google.genai import types
7
8
9  # Load variables from .env
10 load_dotenv()
11
12
13 class GeminiDocumentGenerator:
14     """
15     LegalEase AI Core
16
17     Generates structured legal document drafts using
18     Google's Gemini 3.6 Flash model.
19     """
20
21     def __init__(self):
22         # -----
23         # Load API key
24         # -----
25         self.api_key = os.getenv("GEMINI_API_KEY")
26
27         if not self.api_key:
28             raise ValueError(
29                 "GEMINI_API_KEY is missing from the .env file."
30             )
```

Fig. 12.4 – Gemini AI integration and environment-based model configuration

12.5 Document Formatting and Output Processing

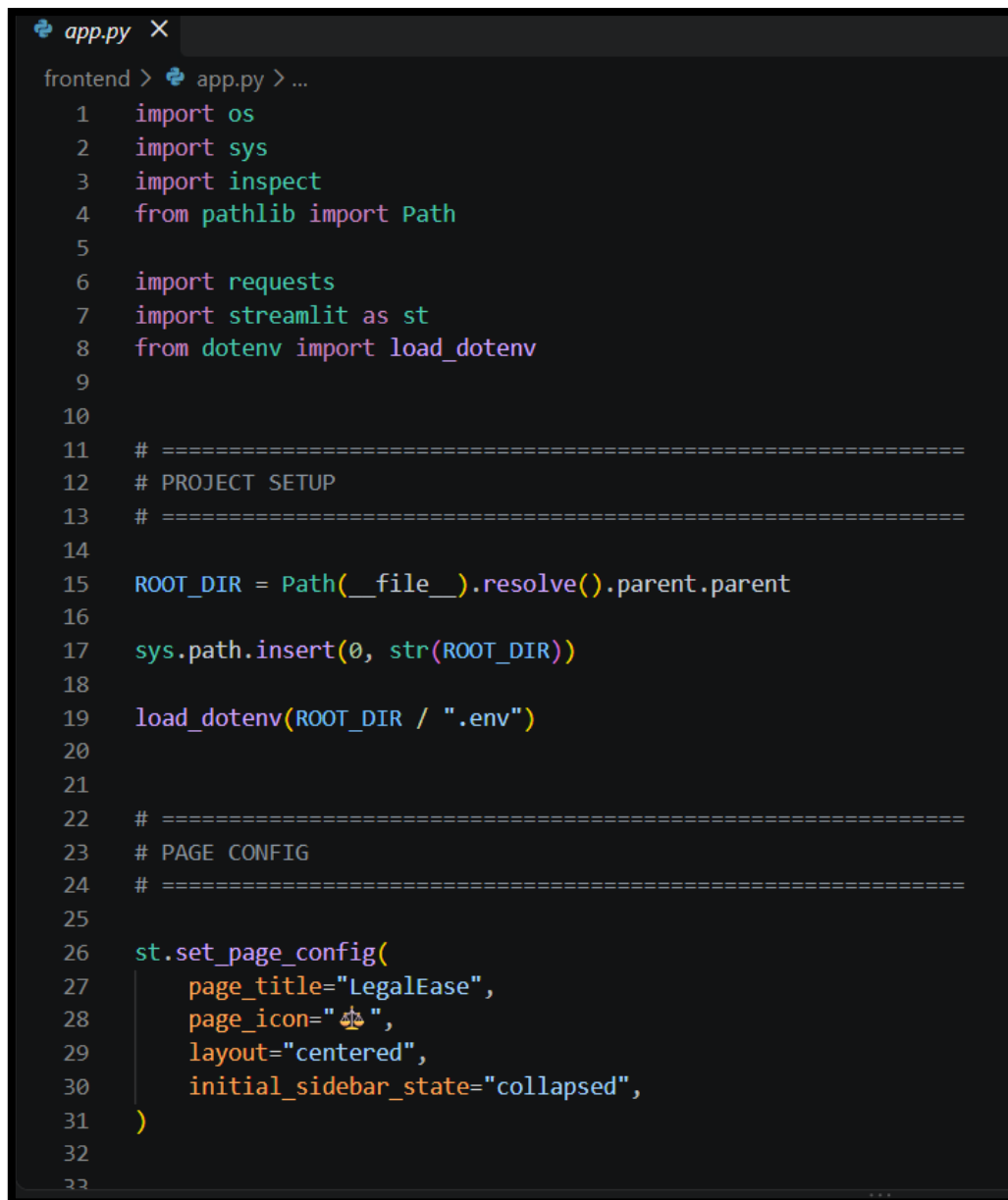
formatter.py is the document-preparation utility. The visible code imports python-docx and ReportLab components and defines `sanitize_text` and `format_document_output`. Sanitization handles empty input safely and strips surrounding whitespace. The formatting function also removes accidental Markdown code fences, helping convert model output into cleaner document-ready text.

```
formatter.py X
utils > formatter.py > format_pdf
1 import io
2 import re
3
4 from docx import Document
5 from reportlab.lib.pagesizes import A4
6 from reportlab.pdfgen import canvas
7
8
9 # =====
10 # SANITIZE
11 # =====
12
13 def sanitize_text(text: str) -> str:
14     if not text:
15         return ""
16
17     return text.strip()
18
19
20
21 # =====
22 # CLEAN MARKDOWN OUTPUT
23 # =====
24
25 def format_document_output(raw_text: str) -> str:
26     if not raw_text:
27         return ""
28
29     text = raw_text.strip()
30
31     # Remove Markdown code fences if Gemini accidentally adds them.
32     text = re.sub(
```

Fig. 12.5 – Text sanitization and document output formatting

12.6 Frontend Application and User Interface

The frontend app.py source initializes the Streamlit application, imports requests and dotenv support, determines the project root, loads the .env file, and configures the Streamlit page. The page is set to a centered layout and starts with the sidebar collapsed. This file is the main presentation-layer entry point.

A screenshot of a code editor window titled 'app.py'. The code is written in Python and shows the initialization of a Streamlit application. It includes imports for 'os', 'sys', 'inspect', 'Path' from 'pathlib', 'requests', 'streamlit' as 'st', and 'load_dotenv' from 'dotenv'. The code sets up the project directory using 'Path(__file__).resolve().parent.parent', inserts it into the system path, and loads the '.env' file. It then configures the Streamlit page with a title 'LegalEase', an icon, a centered layout, and a collapsed sidebar.

```
frontend > app.py > ...
1  import os
2  import sys
3  import inspect
4  from pathlib import Path
5
6  import requests
7  import streamlit as st
8  from dotenv import load_dotenv
9
10
11  # =====
12  # PROJECT SETUP
13  # =====
14
15  ROOT_DIR = Path(__file__).resolve().parent.parent
16
17  sys.path.insert(0, str(ROOT_DIR))
18
19  load_dotenv(ROOT_DIR / ".env")
20
21
22  # =====
23  # PAGE CONFIG
24  # =====
25
26  st.set_page_config(
27      page_title="LegalEase",
28      page_icon="🏛️",
29      layout="centered",
30      initial_sidebar_state="collapsed",
31  )
32
33
```

Fig. 12.6 – Streamlit frontend initialization and page configuration (frontend/app.py screenshot to be inserted)

12.7 User Interface Styling and Theme

The stylesheet defines the application's Luna Blue + Green design system. CSS variables are used for navy, blue, dark blue, teal, green, background, text, muted text, border, and success-state colors. Centralizing these values makes the visual identity easier to maintain consistently across the frontend.

```
# style.css X
frontend > static > # style.css > {} @media (max-width: 700px)
1  /* =====
2  LEGALEASE
3  Luna Blue + Green
4  =====
5
6  :root {
7      --navy: #123b52;
8      --blue: #168bb5;
9      --blue-dark: #116d91;
10     --teal: #28aaa5;
11     --green: #43c69a;
12
13     --background: #f5fbfc;
14     --white: #ffffff;
15
16     --text: #294754;
17     --muted: #718d99;
18
19     --border: #d7e9ec;
20
21     --success-bg: #e7f8ef;
22     --success-border: #c8ead9;
23     --success-text: #13865c;
24 }
25
26
27 /* =====
28 PAGE
29 =====
30
31 .stApp {
32     background:
33     radial-gradient(
```

Fig. 12.7 – Luna Blue + Green frontend styling configuration (style.css screenshot to be inserted)

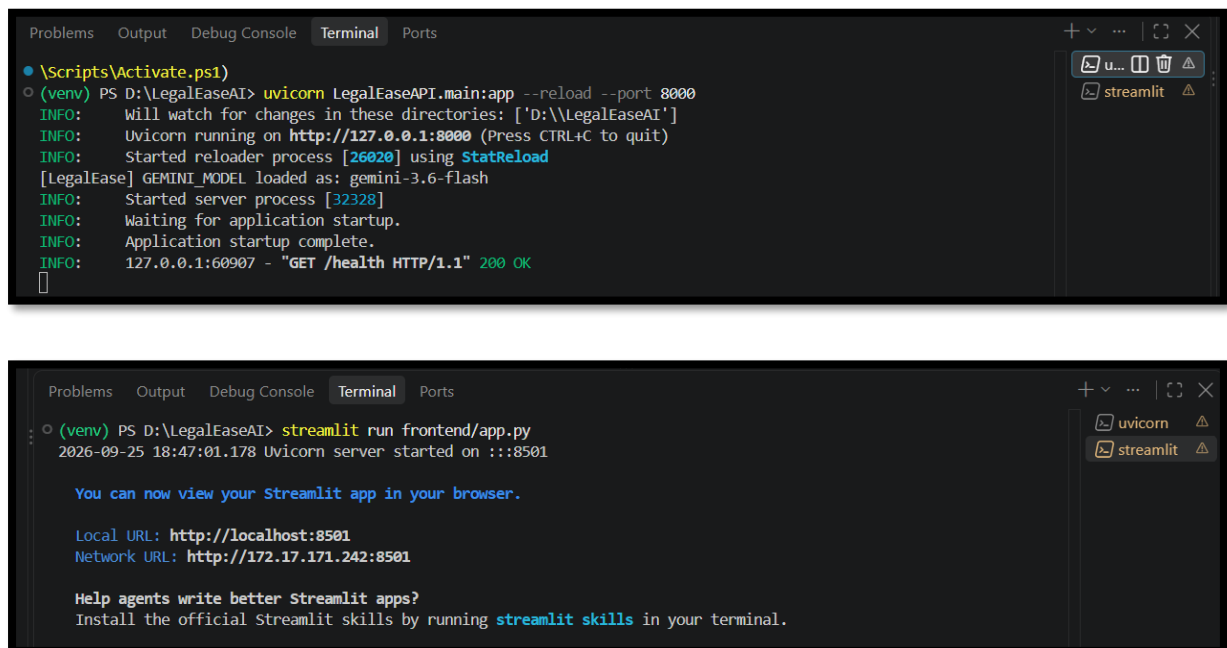
13. Testing Summary

Testing evidence currently available consists primarily of successful application startup and source-level configuration checks. The supplied terminal screenshot shows that python run.py was executed from the activated virtual environment and that the backend and frontend services started without a displayed startup error.

13.1 Startup Test

Expected result: the launcher should start the backend and frontend services and report their local addresses. Observed result: the terminal reports the backend at

127.0.0.1:8000, the Gemini model as loaded, application startup as complete, and the Streamlit frontend at localhost:8501.



```
• \Scripts\Activate.ps1)
○ (venv) PS D:\LegalEaseAI> uvicorn LegalEaseAPI.main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['D:\LegalEaseAI']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [26020] using StatReload
[LegalEase] GEMINI_MODEL loaded as: gemini-3.6-flash
INFO: Started server process [32328]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:60907 - "GET /health HTTP/1.1" 200 OK

○ (venv) PS D:\LegalEaseAI> streamlit run frontend/app.py
2026-09-25 18:47:01.178 Uvicorn server started on :::8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.17.171.242:8501

Help agents write better Streamlit apps?
Install the official Streamlit skills by running streamlit skills in your terminal.
```

Fig. 13.1 – Successful LegalEaseAI service startup shown in the VS Code terminal

13.2 Source-Level Integration Checks

- Backend application creation is visible in main.py.
- Router registration is visible in main.py.
- CORS middleware configuration is visible in main.py.
- Gemini generator initialization and readiness/error handling are visible in routes.py.
- Pydantic request modeling and field constraints are visible in routes.py.
- Environment-based API-key loading is visible in gemini_generator.py.
- Text sanitization and formatting logic are visible in formatter.py.

13.3 Testing Evidence Still Required

For a complete Phase 7 evidence set, browser screenshots should later be added for the home screen, input form, generation action, generated result, and download/output stage. These are deliberately marked as pending rather than being invented.

14. Screenshots / Evidence

The evidence section consolidates the visual proof collected during implementation. Code screenshots demonstrate the internal structure and implementation, while runtime screenshots demonstrate execution. The following evidence is already available:

ID	Evidence	Source	Status	Purpose
E-01	Project structure	VS Code Explorer	Available	Project organization
E-02	Application startup	VS Code terminal	Available	Backend/frontend startup and local addresses
E-03	run.py	Source code	Available	Service startup configuration
E-04	main.py	Source code	Available	FastAPI initialization and router registration
E-05	routes.py	Source code	Available	API routing, validation, Gemini initialization
E-06	gemini_generator.py	Source code	Available	AI service configuration
E-07	formatter.py	Source code	Available	Sanitization and formatting
E-08	frontend/app.py	Source code	Pending screenshot	Streamlit frontend implementation
E-09	style.css	Source code	Pending screenshot	Visual theme implementation
E-10	Running frontend	Browser	Pending screenshot	Actual user interface
E-11	Generated output	Browser	Pending screenshot	End-to-end result

14.1 Screenshot Formatting Standard

- Center each screenshot on the page.
- Preserve the original aspect ratio; never stretch a screenshot.
- Use a thin dark-blue border around screenshots.
- Use a centered, italic dark-green caption directly below each figure.
- Crop unnecessary desktop taskbar or unrelated panels where possible.
- Do not expose API keys, tokens, passwords, or other secrets in screenshots.

15. Installation and Execution

15.1 Project Setup

The project includes a dedicated venv directory, indicating a local Python virtual environment. The project root also contains requirements.txt for dependency declaration and .env/.env.example files for environment configuration.

- Open the LegalEaseAI project folder in Visual Studio Code.
- Create or activate the project's Python virtual environment.
- Install the dependencies listed in requirements.txt.
- Create the required environment configuration based on .env.example.
- Provide the Gemini API key through the environment variable expected by gemini_generator.py.
- Run the application launcher with python run.py.

15.2 Starting the LegalEaseAI Application

The run.py launcher starts the backend and frontend services. The supplied terminal evidence shows the command python run.py running inside the activated virtual environment. The backend is reported on port 8000 and the Streamlit frontend on port 8501.

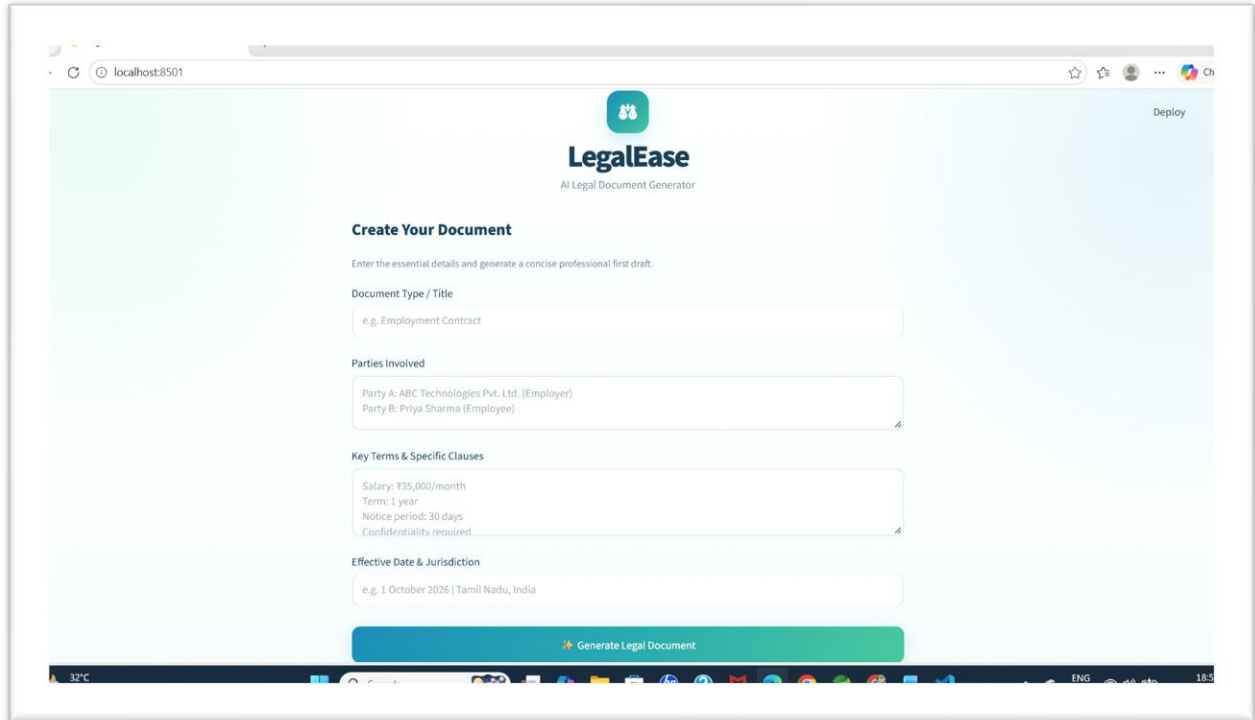


Fig. 15.1 – Successful startup of the LegalEaseAI backend and frontend services

15.3 Accessing the Local Application

Backend address shown by the startup output: <http://127.0.0.1:8000>

Backend health address shown by the startup output: <http://localhost:8000/health>

Frontend address shown by the startup output: <http://localhost:8501>

Security note

Keep the real .env file private. The project should use environment variables for API credentials, and screenshots/documentation should never display the secret value.

16. Project Outcomes

Based on the supplied implementation and runtime evidence, the project achieves a modular application structure with a working local startup process. The backend is configured as a FastAPI service, the frontend is configured as a Streamlit application, and a dedicated Gemini service is integrated into the API layer.

- i. A clear separation between frontend, backend, AI, and formatting responsibilities.
- ii. A single startup script for launching the application services.
- iii. Environment-based configuration for Gemini credentials.

- iv. Structured request modeling and validation.
- v. Explicit AI-service readiness/error state during initialization.
- vi. Text sanitization and document-formatting support.

A defined blue-green visual theme for the frontend.

The end-user outcome should be documented with browser screenshots once available. In particular, the generated legal-document result should be included as the strongest evidence of the complete end-to-end workflow.

17. Future Enhancements

The following enhancements are proposed as logical future work for a project of this architecture. They are presented as development opportunities rather than claims about functionality already implemented.

Add user authentication and role-based access if the application is deployed beyond local use.

Add persistent storage for document history, user requests, and generated files.

Add stronger validation for document-specific fields and domain rules.

Add automated unit and integration tests for API routes, validation, formatting, and AI-service error paths.

Add structured logging and monitoring for production deployments.

Add versioned prompt templates and configurable generation settings.

Add richer document templates and formatting controls.

Add multilingual document generation where appropriate.

Add secure file storage and controlled download permissions.

Add deployment configuration for a cloud environment.

Add browser-level test evidence for the complete user workflow.

18. Conclusion

- LegalEaseAI presents a modular approach to AI-assisted legal document generation. The supplied project evidence demonstrates a clear division between application startup, FastAPI backend initialization, API routing and validation, Gemini integration, document formatting, and Streamlit presentation.
- The implementation is supported by a structured project tree and a successful local startup sequence. The source code also shows environment-based AI configuration, request validation, error-state handling, and document-oriented formatting utilities. Together, these components provide a technical foundation for the intended application workflow.
- For the final Phase 7 submission, the remaining high-value evidence is the running frontend: the user interface, input stage, generation stage, generated document, and download/output stage. Adding those screenshots will complete the visual proof of the end-to-end system and make the documentation substantially stronger.

Final submission

Evidence sequence consistent: Project Structure → Startup → Backend → Routes → Gemini → Formatting → Frontend → User Input → Generated Output → Download/Result.